ML Revision

Synapse AIMLDL Club | STC

Supervised Learning



MACHINE
LEARNING

Prepared by:
Md. Huzaifa

Prepared by:
Parvinkumar S.

mswordcoverpages.com

CURRICULUM ARCHITECTURE & INDEX

Table of Contents & Topic Roadmap

EXECUTIVE OVERVIEW & CURRICULUM SCOPE

This revision handbook compiles the comprehensive Supervised Learning curriculum developed under the Synapse ML/AI Series by the Student Technical Council (STC). Formatted as an intensive single-topic-per-page reference manual, each module provides mathematical intuition, rigorous formulations, diagnostic visualizations, step-by-step numerical worked examples, production-grade scikit-learn code, and community resource links.

CURRICULUM INDEX & MODULE DIRECTORY

MODULE	TOPIC TITLE & ALGORITHMIC SCOPE	SCIKIT-LEARN ESTIMATOR	PAGE
Topic 01	**Simple Linear Regression & Ordinary Least Squares (OLS)** SSE minimization, normal equations, covariance scaling, residual diagnostics	`LinearRegression()`	Page 3
Topic 02	**Multiple Linear Regression, Curve Fitting & Assumptions** Matrix solution $\hat{\beta}=(X^TX)^{-1}X^Ty$, non-linear log curves, R^2, homoscedasticity	`make_pipeline(Poly, OLS)`	Page 4
Topic 03	**Logistic Regression & Maximum Likelihood Estimation** Sigmoid mapping, odds ratios, logit, binary cross-entropy, decision boundaries	`LogisticRegression()`	Page 5
Topic 04	**Classification Evaluation Metrics & Diagnostic Curves** Confusion matrix (TP/FP/FN/TN), Precision, Recall, Specificity, F1, ROC-AUC	`classification_report()`	Page 6
Topic 05	**Decision Trees: Splitting Criteria & Cost-Complexity Pruning** Shannon Entropy, Information Gain, Gini impurity, pre-pruning, cost-complexity α_t	`DecisionTreeClassifier()`	Page 7
Topic 06	**Support Vector Machines (SVM): Geometry, Margins & Kernels** Maximum margin $M=2/\|\|w\|\|$, support vectors, slack ξ_i, C penalty, RBF kernel	`SVC(kernel="rbf")`	Page 8
Topic 07	**Ensemble Methods, Production ML Pipelines & SHAP** Random Forest bagging voting, GBDT boosting, ColumnTransformer, SHAP values	`RandomForestClassifier()`	Page 9

FOUR-PHASE PEDAGOGICAL PROGRESSION PATHWAY

Phase 1: Continuous Regression Foundations (Topics 01–02)
Parametric continuous response modeling, analytical OLS solutions, polynomial expansions, exponential linearization, and residual variance diagnostics.

Phase 2: Probabilistic Classification (Topics 03–04)
Sigmoid probability activations, convex log-loss optimization via MLE, odds ratio interpretation, and holistic confusion matrix / ROC-AUC diagnostic metrics.

Phase 3: Partitioning & Margin Geometry (Topics 05–06)
Recursive information-theoretic feature splitting, tree pruning complexity control, maximum margin hyperplane optimization, and non-linear RBF kernel projections.

Phase 4: Ensembling & Production Systems (Topic 07)
Variance reduction via bagging, bias reduction via gradient boosting, leak-free ColumnTransformer pipelines, and cooperative game-theoretic SHAP explainability.

INSTITUTIONAL METADATA & COMMUNITY RESOURCES

Governing Organization: Synapse (AIMLDL Club) under Student Technical Council (STC).
Authors & Technical Curators: Prepared by Md. Huzaifa & Parvinkumar S.
Community Learning Repository: Accessible via Google Drive: **Synapse Supervised Learning Repository**.
Standard 6-Tier Architecture: Every topic page is structured uniformly across: (1) Conceptual Intuition & Role, (2) Mathematical Derivation, (3) Geometric Visual & Table, (4) Numerical Worked Example, (5) Scikit-Learn Python Pipeline, and (6) Verified Reference Links.

COMMUNITY DRIVE	CURRICULUM DOCUMENT	PEDAGOGICAL FRAMEWORK
Synapse_MachineLearning	Synapse 2-Month Roadmap	7 Core Modules · 9 Complete Pages

Simple Linear Regression & Ordinary Least Squares (OLS)

1. INTUITION & WHY IT IS USED

Intuition: Linear regression models the expected continuous response *y* as an affine function of a predictor *x*. Geometrically, it finds the unique straight line through scattered data points that minimizes the sum of squared vertical prediction errors (residuals).

Why It Is Used: The estimated slope directly quantifies the marginal rate of change in *y* per unit increase in *x*. Ordinary Least Squares provides a closed-form global optimum without requiring iterative learning rates, making it an essential baseline for econometric forecasting and engineering calibration.

2. MATHEMATICAL FORMULATION & ALGORITHMIC DERIVATION

$y_i = a x_i + b + \varepsilon_i$ Predicted: $\hat{y}_i = \hat{a} x_i + \hat{b}$ Residual: $e_i = y_i - \hat{y}_i$ Objective: $\min_{a,b} \sum (y_i - a x_i - b)^2$

$$\text{Optimal Slope: } \hat{a} = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2} = \frac{\text{Cov}(X, Y)}{\text{Var}(X)}$$

Optimal Intercept: $\hat{b} = \bar{y} - \hat{a}\bar{x}$ (Line always passes through centroid $(\bar{x}, \bar{y})$)

Differentiating the residual sum of squares with respect to *a* and *b* and setting partial derivatives to zero yields the Normal Equations: $\sum x_i y_i = a\sum x_i^2 + b\sum x_i$ and $\sum y_i = a\sum x_i + nb$.

3. GEOMETRIC PLOT & RESIDUAL VERIFICATION TABLE

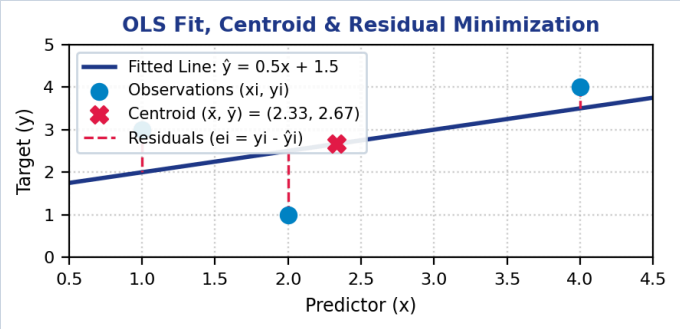


Figure 1.1: OLS Best-Fit Line with Observed Centroid and Residual Discrepancies

OBSERVATION	X_I	Y_I	PREDICTED $\hat{Y}_I$	RESIDUAL E_I	E_I^2
Point 1	1.0	3.0	2.00	+1.00	1.00
Point 2	2.0	1.0	2.50	−1.50	2.25
Point 3	4.0	4.0	3.50	+0.50	0.25
Centroid (x̄, ȳ) = (2.333, 2.667)				**Σ e_i = 0.00**	**SSE = 3.50**

4. STEP-BY-STEP WORKED EXAMPLE

STEP 1: COMPUTE CENTERED CROSS-PRODUCTS

Given points (1, 3), (2, 1), (4, 4) with n = 3.

Means: $\bar{x} = (1+2+4)/3 = 7/3$, $\bar{y} = (3+1+4)/3 = 8/3$.

Centered variance sum: $\Sigma(x_i - \bar{x})^2 = (1-7/3)^2 + (2-7/3)^2 + (4-7/3)^2 = 14/3$.

Covariance sum: $\Sigma(x_i - \bar{x})(y_i - \bar{y}) = (1-7/3)(3-8/3) + (2-7/3)(1-8/3) + (4-7/3)(4-8/3) = 7/3$.

STEP 2: SOLVE CLOSED-FORM COEFFICIENTS

Slope: $\hat{a} = (7/3) / (14/3) = \mathbf{0.50}$.

Intercept: $\hat{b} = 8/3 - (0.50)(7/3) = 8/3 - 7/6 = 9/6 = \mathbf{1.50}$.

Fitted Model: $\hat{y} = 0.50x + 1.50$.

Predictions: $\hat{y}(1) = 2.0$, $\hat{y}(2) = 2.5$, $\hat{y}(4) = 3.5$. Total SSE = 3.50.

5. PYTHON IMPLEMENTATION SNIPPET

```python
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [4]])  # 2D feature matrix (n_samples, n_features)
y = np.array([3, 1, 4])         # 1D target vector

model = LinearRegression().fit(X, y)
print(f"Slope (a): {model.coef_[0]:.2f}, Intercept (b): {model.intercept_:.2f}")  # a = 0.50, b = 1.50
print(f"Fitted Predictions: {model.predict(X)}")  # Output: [2.0, 2.5, 3.5]
```

COMMUNITY DRIVE
SupervisedLearning/LinearRegression

OFFICIAL REFERENCE
Scikit-Learn OLS Documentation

FOUNDATIONAL THEORY
StatQuest Linear Regression Guide

Multiple Linear Regression, Curve Fitting & Assumptions

1. INTUITION & WHY IT IS USED

Intuition: Real-world phenomena depend on multiple concurrent factors and non-linear physical/economic growth. Multiple regression expands features into multidimensional matrix hyperplanes, while non-linear curve fitting linearizes polynomial, exponential, or power models via mathematical transformations without non-convex numerical optimization.

Why It Is Used: Isolates the marginal effect of each predictor while holding confounding variables constant. Goodness of fit is quantified via R^2, and model validity is established by verifying homoscedasticity (constant error variance across predictor domains).

2. MATHEMATICAL FORMULATION & MATRIX MECHANICS

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon} \Rightarrow \hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \quad \text{(Ordinary Least Squares Normal Equation)}$$

Coefficient of Determination: $R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$ Homoscedasticity: $Var(\varepsilon_i | \mathbf{X}) = \sigma^2$

MLE Equivalence: Under independent Gaussian errors $\varepsilon \sim N(0, \sigma^2 I)$, maximizing log-likelihood yields identical coefficients to OLS: $\hat{\beta}_{MLE} = \hat{\beta}_{OLS}$.

3. MODEL COMPARISON & DIAGNOSTICS VISUALIZATIONS

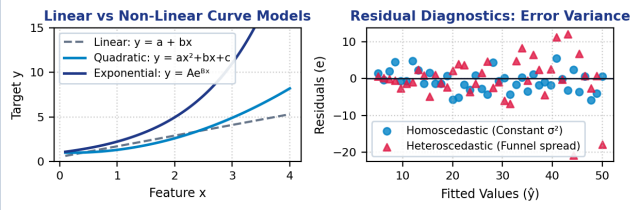


Figure 2.1: Non-Linear Regression Fits and Homoscedastic vs Heteroscedastic Residuals

MODEL	EQUATION	LINEARIZING TRANSFORMATION	PARAMETER RECOVERY
Quadratic	$y = ax^2 + bx + c$	Add x^2 as polynomial feature	Direct OLS solution
Exponential	$y = A\,e^{Bx}$	$\ln(y) = \ln(A) + Bx$	$\hat{A} = e^{\hat{\alpha}}$, $\hat{B}$ = slope
Power Law	$y = A\,x^B$	$\ln(y) = \ln(A) + B\ln(x)$	$\hat{A} = e^{\hat{\alpha}}$, $\hat{B}$ = slope
Multiple OLS	$y = \beta_0 + \sum \beta_j x_j$	Design matrix $[1, X_1, ..., X_p]$	$\hat{\beta} = (X^TX)^{-1}X^Ty$

4. STEP-BY-STEP WORKED EXAMPLE

STEP 1: EXPONENTIAL LINEARIZATION

Given exponential model $y = A\,e^{Bx}$ with observations (0, 2) and (1, 5.436).

Transform target: $Y = \ln(y)$.

Point 1: $(x_1, Y_1) = (0, \ln 2) = (0, 0.6931)$.

Point 2: $(x_2, Y_2) = (1, \ln 5.436) = (1, 1.6931)$.

STEP 2: LINEAR FIT & PARAMETER INVERSION

Slope: $\hat{B} = (1.6931 - 0.6931) / (1 - 0) = \mathbf{1.00}$.

Intercept: $\hat{\alpha} = 0.6931 \Rightarrow \hat{A} = \exp(0.6931) = \mathbf{2.00}$.

Recovered Model: $\hat{y} = 2.00\,e^{1.00x}$.

Test Verification: $\hat{y}(0) = 2.00$, $\hat{y}(1) = 2.00e^1 = 5.436$ ($R^2 = 1.00$).

5. PYTHON IMPLEMENTATION SNIPPET

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import r2_score

# Pipeline: Generate polynomial basis [1, x, x^2] and fit linear regressor
model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(f"Polynomial Model R² Score: {r2_score(y_test, y_pred):.4f}")
```

Logistic Regression & Maximum Likelihood Estimation

1. INTUITION & WHY IT IS USED

Intuition: Linear regression is invalid for classification because linear scores yield impossible predictions outside [0, 1]. Logistic regression maps continuous linear combinations of features through the smooth S-shaped Sigmoid activation function, bounding outputs into well-calibrated posterior probabilities P(Y=1|X) and producing a linear decision boundary in log-odds space.

Why It Is Used: Provides calibrated probabilistic outputs for risk triage (e.g., 82% credit default risk). Exponentiated weights (e^{β}) yield interpretable Odds Ratios, and negative log-likelihood (binary cross-entropy) is strictly convex, ensuring global numerical convergence.

2. MATHEMATICAL FORMULATION & SIGMOID LOGIT DERIVATION

$$\text{Linear Score: } z = \boldsymbol{\beta}^T \mathbf{x} \quad \text{Sigmoid: } p = \sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{Odds} = \frac{p}{1-p} = e^z \quad \text{Logit: } \ln \frac{p}{1-p} = \boldsymbol{\beta}^T \mathbf{x}$$

$$\text{Negative Log-Likelihood (Binary Cross-Entropy Loss): } J(\boldsymbol{\beta}) = -\frac{1}{n} \sum [\, y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i) \,]$$

Odds Ratio Interpretation: e^{β_j} represents the factor by which the odds of the positive class change per unit increase in feature x_j, holding other features constant.

3. SIGMOID PROBABILITY CURVE & SCORE MAPPING TABLE

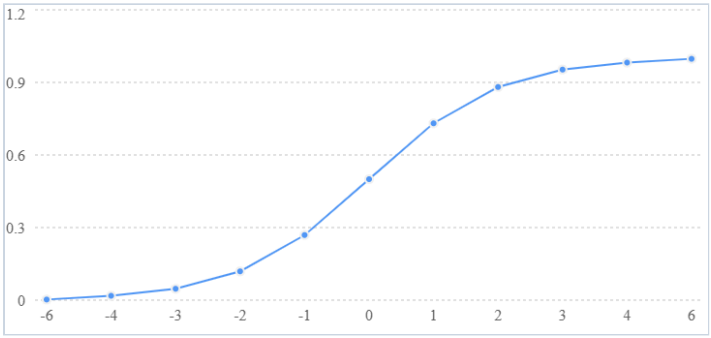


Figure 3.1: Sigmoid Activation Mapping Real Scores $z \in (-\infty, +\infty)$ to Probabilities $p \in [0, 1]$

SCORE Z	SIGMOID P = Σ(Z)	ODDS P/(1−P)	LOG-ODDS LN(ODDS)	PREDICTION (T = 0.5)
−2.00	0.1192	0.1353	−2.00	Class 0
0.00	0.5000	1.0000	0.00	Class 1 (Boundary)
+1.00	0.7311	2.7183	+1.00	Class 1
+ln(3) ≈ 1.10	0.7500	3.0000	+1.10	Class 1
+2.00	0.8808	7.3891	+2.00	Class 1

4. STEP-BY-STEP WORKED EXAMPLE

STEP 1: PROBABILITY CALCULATION

Given linear model score: z = ln(3) ≈ 1.0986.

Sigmoid output: $p = 1 / (1 + e^{-\ln(3)}) = 1 / (1 + 1/3) = \mathbf{3/4 = 0.75}$.

Decision Rule at cutoff t = 0.5: **Predicted Class = 1** (since 0.75 ≥ 0.5).

STEP 2: LOG-LOSS PENALTY EVALUATION

• Actual Label y = 1: Loss = −ln(0.75) ≈ **0.2877**.

• Actual Label y = 0: Loss = −ln(1 − 0.75) = −ln(0.25) ≈ **1.3863**.

Overconfident incorrect predictions are penalized exponentially.

5. PYTHON IMPLEMENTATION SNIPPET

```python
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# Robust scaled pipeline: Standardization ensures gradient-based optimization stability
pipe = make_pipeline(StandardScaler(), LogisticRegression(C=1.0, max_iter=1000))
pipe.fit(X_train, y_train)

y_prob = pipe.predict_proba(X_test)[:, 1]  # Posterior probabilities P(Y=1|X)
y_pred = (y_prob >= 0.5).astype(int)       # Threshold decision boundary at t = 0.5
```

Classification Evaluation Metrics & Diagnostic Curves

1. INTUITION & WHY IT IS USED

Intuition: Accuracy is deceptive in imbalanced domains: a model predicting all negative achieves 99% accuracy on a 1% fraud rate while detecting zero fraud. Rigorous evaluation deconstructs predictions into a Confusion Matrix, assessing trade-offs between false alarms (Type I) and missed detections (Type II) across decision thresholds.

Why It Is Used: Aligns model tuning with clinical and financial stakes (e.g., maximizing recall in disease screening vs precision in spam filtering) and provides threshold-independent ROC-AUC benchmarking.

2. MATHEMATICAL FORMULATION & METRIC FRACTIONS

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad \text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall (TPR)} = \frac{TP}{TP + FN} \quad \text{Specificity} = \frac{TN}{TN + FP}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad \text{(Harmonic mean penalizing extreme drops in either metric)}$$

$$\text{FPR} = 1 - \text{Specificity} = \frac{FP}{FP + TN}$$

ROC-AUC vs PR-AUC: ROC-AUC measures true ranking separation across all possible thresholds; PR-AUC is essential when positive prevalence is rare (<5%).

3. CONFUSION MATRIX HEATMAP & ROC CURVE

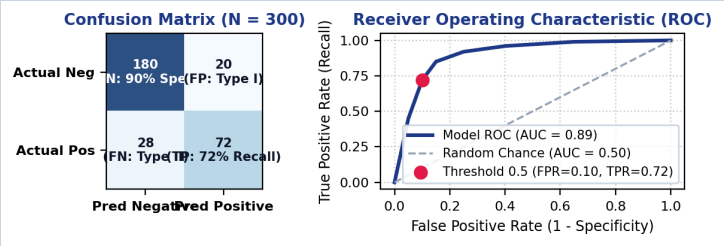


Figure 4.1: Confusion Matrix Grid (N = 300) and Receiver Operating Characteristic Curve

METRIC	EXACT FORMULA	APPLICATION PRIORITY	TARGET ERROR
Precision	TP / (TP + FP)	Spam filter, fraud block (costly false alarms)	Minimizes False Positives
Recall	TP / (TP + FN)	Cancer screening, defect detection (fatal misses)	Minimizes False Negatives
F1-Score	2PR / (P + R)	Balanced benchmark under moderate imbalance	Harmonizes Purity & Coverage
ROC-AUC	Area under TPR vs FPR	Global classifier capability across all thresholds	Ranking discrimination

4. STEP-BY-STEP WORKED EXAMPLE

STEP 1: EVALUATION COUNTS

Dataset: TP = 72, TN = 180, FP = 20, FN = 28.
Total N = 72 + 180 + 20 + 28 = 300.
Accuracy = (72 + 180) / 300 = 252 / 300 = **84.00%**

STEP 2: METRIC CALCULATIONS

Precision = 72 / (72 + 20) = 72 / 92 ≈ **78.26%**
Recall = 72 / (72 + 28) = 72 / 100 = **72.00%**
Specificity = 180 / (180 + 20) = 180 / 200 = **90.00%**
F1-Score = 2(0.7826 · 0.72) / (0.7826 + 0.72) ≈ **75.00%**

5. PYTHON IMPLEMENTATION SNIPPET

```
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score

print("Confusion Matrix:
", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, digits=4))
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}")  # Continuous decision scores
```

COMMUNITY DRIVE
Synapse_MachineLearning/Evaluation

OFFICIAL REFERENCE
Scikit-Learn Evaluation Metrics

FOUNDATIONAL THEORY
Google ML Crash Course

Decision Trees: Splitting Criteria & Cost-Complexity Pruning

1. INTUITION & WHY IT IS USED

Intuition: Decision Trees perform recursive binary partitioning, dividing the feature space into orthogonal rectangular regions using hierarchical IF-THEN rules. Rather than estimating continuous hyperplanes, the tree iteratively selects features and split thresholds that maximize child class purity.

Why It Is Used: Fully transparent and interpretable as flowchart logic for medical triage and lending compliance. Requires zero monotonic feature scaling, natively ingests numerical and categorical features, and automatically discovers complex non-linear feature interactions.

2. MATHEMATICAL FORMULATION & IMPURITY REDUCTIONS

$$\text{Shannon Entropy: } H(S) = -\sum p_k \log_2(p_k) \quad \text{Gini Impurity: } G(S) = 1 - \sum p_k^2 = 2p(1-p) \quad IG(S,A) = H(S) - \sum \frac{|S_j|}{|S|} H(S_j)$$

$$\text{Cost-Complexity Post-Pruning: } R_\alpha(T) = R(T) + \alpha|T| \quad \Rightarrow \quad \text{Effective Alpha: } \alpha_t = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

Pruning Mechanics: Pre-pruning limits growth early (max_depth, min_samples_leaf); Cost-complexity post-pruning collapses subtrees whose error reduction does not justify their leaf count penalty α.

3. DECISION TREE HIERARCHY & SPLITTING CRITERIA TABLE

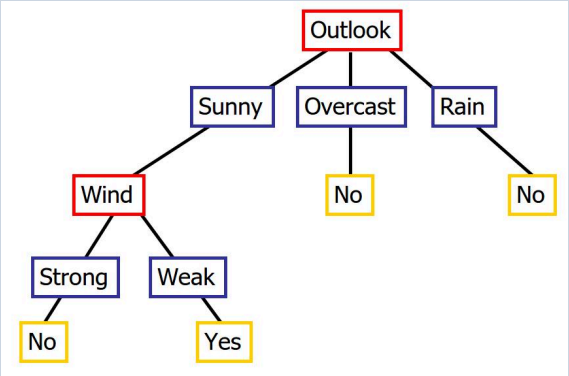


Figure 5.1: Decision Tree Architecture (Outlook & Wind Partitioning)

CRITERION	MATHEMATICAL FORMULA	BINARY RANGE	COMPUTATIONAL COST
Entropy	$-\sum p_k \log_2(p_k)$	0.0 to 1.0 bit	Higher (Logarithmic ops)
Gini Impurity	$1 - \sum p_k^2$	0.0 to 0.50	Lower (Square ops only)
Misclass. Error	$1 - \max(p_k)$	0.0 to 0.50	Lowest (Not smooth for splits)
Cost-Complexity	$R(T) + \alpha\|T\|$	$\alpha \geq 0$	Tuned via cross-validation

4. STEP-BY-STEP WORKED EXAMPLE

STEP 1: COMPUTE PARENT UNCERTAINTY

Parent node S: 10 samples (5 Positive, 5 Negative).
$H(S) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) =$ **1.000 bit**.
Candidate split creates Child A (4 pos, 1 neg) and Child B (1 pos, 4 neg), both size 5.

STEP 2: INFORMATION GAIN EVALUATION

$H(A) = H(B) = -0.8 \log_2(0.8) - 0.2 \log_2(0.2) \approx$ **0.7219 bits**.
$H_{after} = (5/10)(0.7219) + (5/10)(0.7219) = 0.7219$ bits.
Information Gain (IG) = 1.000 − 0.7219 = **0.2781 bits**.

5. PYTHON IMPLEMENTATION SNIPPET

```
from sklearn.tree import DecisionTreeClassifier

# Build pre-pruned tree to prevent high-variance memorization
tree = DecisionTreeClassifier(criterion="gini", max_depth=4, min_samples_leaf=5, random_state=42)
tree.fit(X_train, y_train)

# Extract minimal cost-complexity pruning path for alpha optimization
path = tree.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas
print(f"Pruned Depth: {tree.get_depth()} | Leaf Nodes: {tree.get_n_leaves()}")
print(f"Optimal alpha range: [{ccp_alphas.min():.5f}, {ccp_alphas.max():.5f}]")
```

COMMUNITY DRIVE
DecisionTree & Ensembles PDF

OFFICIAL REFERENCE
Scikit-Learn Decision Trees

FOUNDATIONAL THEORY
NYU Machine Learning Lecture Notes

Support Vector Machines (SVM): Geometry, Margins & Kernels

1. INTUITION & WHY IT IS USED

Intuition: Support Vector Machines identify the unique separating hyperplane that maximizes the geometric margin between classes. The boundary is determined exclusively by the critical data points lying closest to the boundary—the *Support Vectors*. Points lying farther away exert zero influence on the decision surface.

Why It Is Used: Exceptional generalization in high-dimensional feature spaces (genomics, text embeddings). The hyperparameter C controls regularization against noisy outliers, and the Kernel Trick enables non-linear separation without explicit coordinate transformation.

2. MATHEMATICAL FORMULATION & SOFT-MARGIN OPTIMIZATION

Hyperplane: $\mathbf{w}^T\mathbf{x} + b = 0$ Point Distance: $d(\mathbf{x}_0) = \frac{|\mathbf{w}^T\mathbf{x}_0 + b|}{\|\mathbf{w}\|}$ Margin Width: $M = \frac{2}{\|\mathbf{w}\|}$

Soft-Margin Primal Objective: $\min_{\mathbf{w}, b, \xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum \xi_i$ s.t. $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$

Regularization Trade-off: Large C enforces narrow margins with strict violation penalties (risk of overfitting); Small C permits slack violations for wider margins (higher regularization).

3. MARGIN GEOMETRY & KERNEL SEPARATION COMPARISON

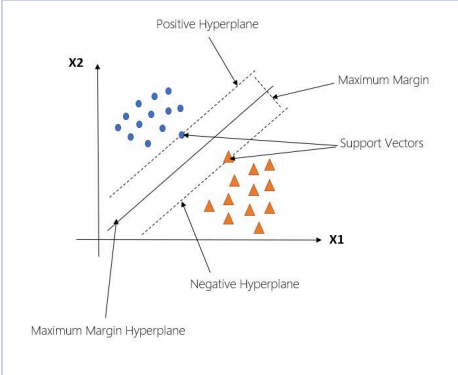


Figure 6.1: Maximum Margin Hyperplane, Supporting Boundaries, and Support Vectors

KERNEL TYPE	MATHEMATICAL FUNCTION K(X, Z)	HYPERPARAMETERS	RECOMMENDED DOMAIN
Linear	x^Tz	C	High-dimensional text, genomics
RBF (Gaussian)	$\exp(-\gamma \|x - z\|^2)$	C, γ (gamma)	General non-linear default
Polynomial	$(\gamma x^Tz + r)^d$	C, γ, degree d, r	Physical power interactions
Sigmoid	$\tanh(\gamma x^Tz + r)$	C, γ, r	Neural network approximations

4. STEP-BY-STEP WORKED EXAMPLE

STEP 1: MARGIN & DISTANCE CALCULATION

Given: w = (3, 4) and b = −10.

Weight Norm: $\|w\| = \sqrt{(3^2 + 4^2)} = \mathbf{5.0}$.

Margin Width: M = 2 / ||w|| = 2 / 5 = **0.40 units**.

Distance of point (2, 3): d = |3(2) + 4(3) − 10| / 5 = 8 / 5 = **1.60**.

STEP 2: SLACK VARIABLE ANALYSIS

Observation signed margin: $m = y(w^Tx + b) = 0.40$.

Slack: $\xi = \max(0, 1 - m) = \max(0, 1 - 0.40) = \mathbf{0.60}$.

Status: Inside margin ($0 < \xi < 1$) but correctly classified ($m > 0$).

Total penalty contribution = 0.60C.

5. PYTHON IMPLEMENTATION SNIPPET

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# Pipeline: Standardization is strictly required for distance-dependent SVM
svm_clf = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1.0, gamma="scale"))
svm_clf.fit(X_train, y_train)

scores = svm_clf.decision_function(X_test)  # Continuous distance scores for ROC analysis
print(f"Support Vectors per Class: {svm_clf.named_steps['svc'].n_support_}")
```

COMMUNITY DRIVE
Synapse_MachineLearning/DecodingSVM

OFFICIAL REFERENCE
Scikit-Learn SVM Documentation

FOUNDATIONAL THEORY
Stanford CS229 SVM Formulation

Ensemble Methods, Production ML Pipelines & SHAP

1. INTUITION & WHY IT IS USED

Intuition: Harnesses the "wisdom of crowds": combining diverse individual learners yields predictive stability and accuracy superior to any single model. Random Forests build independent randomized trees in parallel to drive down variance (bagging), while Gradient Boosting builds trees sequentially to iteratively correct the residual errors of preceding trees (boosting).

Why It Is Used: Tree ensembles dominate competitive tabular benchmarks (Kaggle, financial risk). Unified pipelines (ColumnTransformer) guarantee strict isolation between training and test sets, and SHAP ensures game-theoretic interpretability.

2. MATHEMATICAL FORMULATION & ATTRIBUTION MECHANICS

Random Forest: $\hat{y} = \text{mode}\{h_b(x)\}$ or $\frac{1}{B}\sum h_b(x)$ Gradient Boosting Residual: $r_{im} = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$

Ensemble Step Update: $F_m(x) = F_{m-1}(x) + \eta f_m(x)$ (η = learning rate shrinkage)

SHAP Attribution: $f(x) = \varphi_0 + \sum \varphi_j(x)$

Leak-Free Pipeline: Raw data split first ⇒ ColumnTransformer fits imputation, encoding, and scaling strictly on training folds.

3. ARCHITECTURE COMPARISON & SHAP WATERFALL PLOT

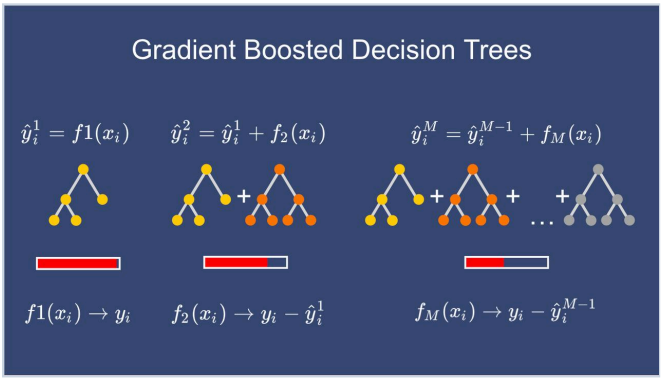


Figure 7.1: Sequential Residual Fitting in Gradient Boosted Decision Trees

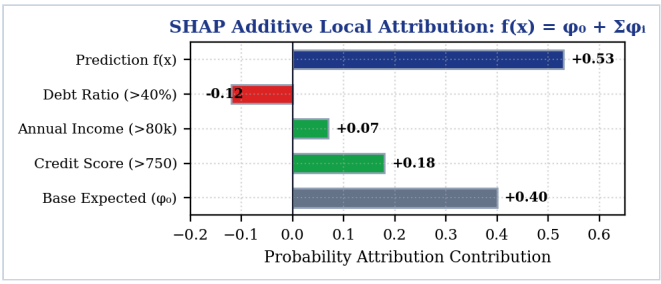


Figure 7.2: Game-Theoretic SHAP Additive Feature Attributions

4. STEP-BY-STEP WORKED EXAMPLE

STEP 1: BASELINE EXPECTATION

Trained Loan Approval Classifier.

Base dataset approval expectation: $\varphi_0 = \mathbf{0.40}$ (40%).

Applicant Feature Vector evaluated.

STEP 2: ADDITIVE SHAP ATTRIBUTION

- High Credit Score (>750): $\varphi_1 = \mathbf{+0.18}$
- Annual Income (>80k): $\varphi_2 = \mathbf{+0.07}$
- Debt-to-Income (>40%): $\varphi_3 = \mathbf{-0.12}$

Model Prediction f(x) = 0.40 + 0.18 + 0.07 − 0.12 = **0.53 (53%)**

5. PYTHON IMPLEMENTATION SNIPPET

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

# Leakage-Safe End-to-End Pipeline: Fit strictly on training folds
num_pipe = Pipeline([('imp', SimpleImputer(strategy='median')), ('scl', StandardScaler())])
cat_pipe = Pipeline([
    ('imp', SimpleImputer(strategy='most_frequent')),
    ('enc', OneHotEncoder(handle_unknown='ignore'))
])
prep = ColumnTransformer([('num', num_pipe, num_cols), ('cat', cat_pipe, cat_cols)])
model = Pipeline([('prep', prep), ('rf', RandomForestClassifier(n_estimators=100, random_state=42))])
model.fit(X_train, y_train)
```

COMMUNITY DRIVE
Synapse_MachineLearning/Ensembles_Pipelines

OFFICIAL REFERENCE
Scikit-Learn Ensemble Guide

FOUNDATIONAL THEORY
SHAP NeurIPS Documentation